

Segment-Level Agentic Topic Modeling for Improved Data Exploration and Resource Efficiency

Myeongjun Erik Jang¹ Antonios Georgiadis¹ Sae Young Moon¹ Fran Silavong¹

Abstract

Topic modeling is an effective technique for discovering hidden themes within documents and is widely used in text mining and data analysis across a variety of industry sectors. Recently, large language model (LLM)-based topic models have emerged that prompt LLMs to generate topics then assign the topics to documents, producing more natural and human-readable topics than conventional topic modeling algorithms. However, the nature of topic assignment process causes certain drawbacks, such as the incapability to produce topic distributions over a document, too broad or narrow topics, and high resource consumption, which increases with the number and length of documents being assigned topics. These issues are particularly critical for industrial applications, which require high-quality, in-depth analysis and the processing of large volumes of documents. In this context, this paper introduces a framework called SeLATM, which addresses these concerns by employing segment-level topic generation and topic refinement through agentic feedback loops. Experimental results on various datasets demonstrate that SeLATM significantly reduces the LLM resources compared to methods based on topic assignment process, while maintaining superior performance.

1. Introduction

Topic modeling is a Natural Language Processing (NLP) technique designed to discover meaningful topics within a corpus, which has a broad range of practical usage for data explorations in research and industry (Ranaei et al., 2017; Asmussen & Møller, 2019; Xiong et al., 2019). Recent advances in large language models (LLMs) have further

propelled progress in this area. Compared to conventional methods such as LDA (Blei et al., 2003), which represent a topic as distributions over words, these approaches prompt LLMs to generate free-text topics and assign the topic to individual documents (Pham et al., 2024; Lam et al., 2024; Mu et al., 2024; Doi et al., 2024; Liu et al., 2025; Moon et al., 2026), resulting in more comprehensive and human-understandable topics.

Although, LLM-driven topic generation and assignment methods provide significant benefits for the intuitiveness of topic representations, they are limited in their ability to capture document-topic relations. It is natural for a document to cover multiple topics. For example, a monthly financial report may address topics such as interest rates, global events, and political changes, with varying amounts of content devoted to each topic. Conventional topic modeling methods can capture this information, as they represent each document as a distribution over multiple topics. In contrast, LLM-based approaches typically assign only a single topic to each document (Lam et al., 2024; Liu et al., 2025; Moon et al., 2026); even when multi-topic assignment is supported, these methods cannot precisely quantify how much each topic contributes to a document, limiting interpretability for data exploration. Also, the topic assignment process is highly resource-intensive, as it requires labeling topics to each individual document. Consequently, the LLM call cost $O(N \times L)$ where N and L imply the number of documents and the document length. This poses a significant challenge for real-world industrial applications that process large volumes of data, particularly in sectors handling long documents, such as finance (Masry & Hajian, 2024; Hu et al., 2025) and legal (Chalkidis et al., 2022; Jang & Stikkel, 2024).

Furthermore, current LLM-based topic modeling approaches pay limited attention to topic refinement, either neglecting it at all (Mu et al., 2024; Doi et al., 2024) or focusing solely on topic merging to reduce overlap (Pham et al., 2024; Liu et al., 2025; Moon et al., 2026). However, there is a potential risk that the generated topics maybe less coherent and could be further divided into smaller, more distinct topics. Additionally, relying solely on topic merging can result in ambiguous and vague topics. For example, consider two

¹J.P. Morgan Chase, London, United Kingdom. Correspondence to: Myeongjun Erik Jang <myeongjun.jang@jpmorganchase.com>, Fran Silavong <fran.silavong@jpmorganchase.com>.

topics, T_A and T_B , each comprising several subtopics (t_i): $T_A = \{t_i, 1 \leq i \leq 5\}$ and $T_B = \{t_i, 3 \leq i \leq 7\}$, respectively. Let us assume that the existing topic-merging methods combine these topics due to a significant overlap of 60%. This results in a mega-topic $T = \{t_i, 1 \leq i \leq 7\}$, which could be too broad and incoherent, especially if subtopics t_1 and t_2 are distinct from t_6 and t_7 . Instead, it would be more natural to separate t_3, t_4, t_5 from T_A and T_B , create a new topic, and then decide whether to merge it with the closer of T_A or T_B ; if not, the new topic should remain separate.

To this end, we propose a novel LLM-driven agentic topic modeling algorithm called SELATM (**Segment Level Agentic Topic Modelling**). As we regarded topic assignment as a major bottleneck of modern LLM-driven topic modeling, we remove this step from our pipeline. Instead, to facilitate accurate topic naming from the outset, we introduce segment-level topic modeling, in which documents are divided into segments and topic names are assigned at the level of segment clusters. Owing to the semantic conciseness of text segments and their benefit to alleviate LLMs’ limitations when handling long contexts (Liu et al., 2024; Zhou et al., 2024), our method can produce more precise topic names than previous approaches. In addition, our framework represents documents as distributions over multiple topics, allowing the contribution of each topic to a document and the association of specific content with corresponding topics, while also reducing computational costs. Furthermore, inspired by recent progress in iterative refinement via LLM-driven feedback loops (Madaan et al., 2023; Yuksel et al., 2025; Ravi et al., 2025), we propose an agentic topic refinement process. In this framework, evaluator agents assess the quality of generated topics, operator agents apply topic split and merge operations based on the feedback from the evaluator, and a planner agent determines whether additional refinement iterations are required. To the best of our knowledge, this work is the first to explore the use of refinement-based feedback loops for topic modeling. Prior LLM-based topic modeling approaches have been less amenable to such iterative refinement, as repeated topic assignment can substantially increase computational cost. Experiments on multiple public and industry datasets suggest that SELATM yields higher-quality topics while remaining resource-efficient, thereby helping to mitigate the aforementioned concerns.

2. SELATM Framework

2.1. Segment-level Topic Modeling

Segmentation and Clustering. Unlike a document that often spans multiple topics, a sentence or paragraph is typically structured to focus on a single core topic—a fundamental principle of written composition (Strunk Jr & White, 2007; Moens, 2008; Halliday & Matthiessen, 2013). This

background motivates the following intuitions that underpin our approach:

1. As sentence- or paragraph-level segments will concentrate on a single topic, a cluster of semantically similar segments is expected to address the common topic underlying those segments.
2. Assigning a label to a text cluster that conveys a single topic is considerably more straightforward than labeling a document that addresses multiple topics, whether using single or multiple labels.
3. The reduced task complexity enables to achieve higher topic labeling accuracy, eliminating the need for document-level topic assignment process.

Additionally, reduced token lengths offer several practical benefits: they help mitigate the lost-in-the-middle problem (Liu et al., 2024), a well-known issue in long-context scenarios. Furthermore, it helps avoid length collapse (Zhou et al., 2024)—a phenomenon in which embeddings of long texts tend to cluster together and become less distinguishable—resulting in more precise clustering outcomes.

Let D_i represent the i -th document in a corpus $\mathcal{D} = \{D_1, D_2, \dots, D_N\}$. SELATM first separates D_i into text segments, $D_i = \{s_{i1}, s_{i2}, \dots, s_{iM}\}$, where s_{ij} denotes the j -th segment of D_i . Specifically, SELATM adopts commonly used granularity for segmentation, sentence- and paragraph-level (Gao et al., 2023), which can be generated using syntactic rules and do not require LLM usage. We also experimented with recent semantic-based segmentation methods, such as LUMBERCHUNKER (Duarte et al., 2024); however, these approaches rely heavily on LLMs, which conflicts with our resource-efficiency objectives. Moreover, as sentences and paragraphs are typically organized around a single dominant topic, as mentioned above, we considered syntactic segmentation to be sufficient for our purposes. After segmentation, embeddings are generated for each text segment and subsequently clustered:

$$\begin{aligned} e_{ij} &= \text{Emb}(s_{ij}), & \mathcal{C} &= \{C_1, C_2, \dots, C_K\} \\ f_c(e_{ij}) &\mapsto C_k \in \mathcal{C}, \\ C_k &= \{s_{ij} \mid f_c(e_{ij}) = k\} \end{aligned}$$

where e_{ij} denotes the embedding vector of s_{ij} generated by the embedding model Emb , C_k is k -th segment cluster, and f_c is the clustering function that assigns e_{ij} into its corresponding cluster. We used agglomerative clustering (Murtagh & Legendre, 2014), as it produced a more stable and robust performance compared to K-means. The number of clusters (K) is a user-defined hyperparameter; increasing its value results in more granular topics. K is treated as a granularity control rather than a ground-truth estimate, with downstream refinement mitigating moderate mis-specification.”

Topic Generation. The subsequent step involves generat-

ing topics for each text segment cluster. Regarding large size clusters, incorporating all text segments for topic generation is computationally inefficient. Therefore, to such clusters, SELATM leverages Maximal Marginal Relevance (MMR, Carbonell & Goldstein 1998) sampling, which effectively extracts representative examples through an iterative selection process that balances the relevance and diversity of the selected documents (Pattnaik et al., 2024):

$$\tilde{C}_k = \begin{cases} C_k & \text{if } |C_k| \leq n_S, \\ \text{Sample}(C_k, n_S) & \text{otherwise,} \end{cases}$$

where n_S is a hyperparameter that specifies the number of text segments sampled for topic generation. Finally, given the text segments of each cluster, a topic generation agent ($\mathcal{A}_{TG}$) generates topic names along with corresponding descriptions, thereby improving topic interpretability:

$$(t_k, d_k) = \mathcal{A}_{TG}(\tilde{C}_k), \\ (t_k, d_k) \leftrightarrow C_k = \{s_{ij} \mid f_c(s_{ij}) = k\},$$

where t_k and d_k denote the topic name and description of cluster k . Sampling is applied exclusively before LLM calls and never before clustering or topic distribution computation. The prompt for topic generation is presented in Appendix B.1.

As a result, SELATM can represent a document D_i as a distribution over topics as follows, where p_{ik} is the contribution of t_k to D_i :

$$D_i = (p_{i1}, p_{i2}, \dots, p_{iK}) \quad \text{where,} \\ p_{ik} = \frac{1}{|D_i|} \sum_{j=1}^M 1(f_c(s_{ij}) = C_k).$$

This enables us to capture the contribution of each topic and identify which segments are associated with certain topic—a capability not offered by previous LLM-driven topic modeling methods. In addition, an LLM is invoked $O(K \times n_S \times l)$ times, where l is the length of the text segments. This is a significant reduction from $O(N \times L)$, given that $K \times n_S$ and l are much smaller than N and L , respectively.

2.2. Agentic Topic Refinement

Inspired by recent advances in leveraging LLMs to generate textual feedback (Yuksekgonul et al., 2024; Yin & Wang, 2025) and incorporate it into iterative refinement loops (Madaan et al., 2023; Yuksel et al., 2025; Ravi et al., 2025), we introduce an iterative topic refinement feedback loop that various agents collaborate. Although some applications demonstrate successful fully LLM-driven process planning (Song et al., 2023), more comprehensive studies indicate that LLMs often perform poorly as planners (Valmeekam et al., 2022; 2023; Kambhampati,

2024; Dagan et al., 2024), and recommend manual curation of plans, roles, and prompts for a task requiring consistency (Sypherd & Belle, 2024). Hence, we designed a refinement process, which is presented in Algorithm 1.

Topic Coherence Evaluation. The first step is to measure topic coherence, which assess how closely the generated topic name and description are related to the text segments assigned to that topic. Low coherence indicates the present of text segments that do not align well with the assigned topic, suggesting that the topic may need to be separated. For each sampled segment cluster ($\tilde{C}_k$), the coherence evaluation agent ($\mathcal{E}_{coh}$) evaluates topic coherence in parallel using the cluster’s segments, topic name, and description. If a topic is found to have low coherence, $\mathcal{E}_{coh}$ marks it as needing separation and provides feedback on how coherence can be improved. Detailed information regarding the prompt design of $\mathcal{E}_{coh}$ and its output format is provided in Appendix B.2.

Topic Split Operation. For topics identified as needing separation, the split operation agent ($\mathcal{A}_{split}$) performs the separation. Specifically, the text segments of the topic (C_k) are re-clustered with the number of clusters K set to 2. Please note that C_k is used instead of $\tilde{C}_k$, because segments that are not included in the sampled segment cluster must be assigned to one of the new topics. Subsequently, we measure the proportion of text segments in the larger of the two separated clusters. If this proportion exceeds certain threshold (80% in our experiments), we skip the split operation, as this indicates that, despite $\mathcal{A}_{split}$ recommending separation, the majority of text segments still pertain to a single topic. Otherwise, $\mathcal{A}_{TG}$ regenerates the topic names and descriptions for each separated segment cluster, this time incorporating the feedback provided by $\mathcal{E}_{coh}$ into the prompt. Once the names and descriptions are re-generated, we update $\tilde{C}$ accordingly by removing the split topic and adding the new topics resulting from the separation.

Topic Diversity Evaluation. After the split operation, topic diversity is assessed by topic diversity evaluation agent ($\mathcal{E}_{div}$), which evaluates how distinctive each topic is from the others. This is a holistic, one-shot evaluation that takes all topic names and descriptions ($\mathcal{T}_g$) as input and identifies which topic groups should be merged, returning the corresponding topic indices along with feedback on how merging can improve topic diversity. Detailed information about the prompt used for $\mathcal{E}_{div}$ and its output format is provided in Appendix B.3.

Topic Merging Operation. Given that $\mathcal{E}_{div}$ identifies the need for topic merging and specifies the groups of topics to be united, the merge operation agent ($\mathcal{A}_{merge}$) combines the sampled text segments ($\tilde{C}_k$) of each identified topic group and run $\mathcal{A}_{TG}$ to generate the name and description of the merged topic. As with the topic split operation, feedback

Algorithm 1 Agentic Topic Refinement Feedback Loop

Input: generated topics and descriptions $\mathcal{T}_g = \{(t_1, d_1), (t_2, d_2), \dots, (t_K, d_K)\}$, segment clusters $\mathcal{C} = \{C_1, C_2, \dots, C_K\}$, sampled segment clusters $\tilde{\mathcal{C}} = \{\tilde{C}_1, \tilde{C}_2, \dots, \tilde{C}_K\}$, number of iterations e , maximum tolerance T .

Agents: topic coherence evaluator agent $\mathcal{E}_{coh}$, topic diversity evaluator agent $\mathcal{E}_{div}$, split operation agent $\mathcal{A}_{split}$, merge operation agent $\mathcal{A}_{merge}$, planner agent $\mathcal{A}_{plan}$

Output: refined $\mathcal{C}$ and $\mathcal{T}_g$.

Initialize $tolerance = 0$.

for $i = 1$ **to** e **do**

if $tolerance > T$ **then**

break

end if

$\mathcal{C}^i, \tilde{\mathcal{C}}^i, \mathcal{T}_g^i = \text{COPY}(\mathcal{C}, \tilde{\mathcal{C}}, \mathcal{T}_g)$

Generate coherence evaluation feedback

$Coh_{fb} = \mathcal{E}_{coh}(\tilde{\mathcal{C}}^i, \mathcal{T}_g^i)$

Run split operation, and update $\mathcal{C}^i$, $\tilde{\mathcal{C}}^i$, and $\mathcal{T}_g^i$

$\mathcal{C}^i, \mathcal{T}_g^i = \mathcal{A}_{split}(Coh_{fb}, \mathcal{C}^i, \mathcal{T}_g^i)$

$\tilde{\mathcal{C}}^i = \text{UPDATE}(\tilde{\mathcal{C}}^i, \mathcal{C}^i)$

Generate diversity evaluation feedback

$Div_{fb} = \mathcal{E}_{div}(\mathcal{T}_g^i)$

Run merge operation, and update $\mathcal{C}^i$, $\tilde{\mathcal{C}}^i$, and $\mathcal{T}_g^i$

$\tilde{\mathcal{C}}^i, \mathcal{T}_g^i = \mathcal{A}_{merge}(Div_{fb}, \tilde{\mathcal{C}}^i, \mathcal{T}_g^i)$

$\mathcal{C}^i = \text{UPDATE}(\tilde{\mathcal{C}}^i, \mathcal{C}^i)$

Re-evaluate refined topics

$Coh_{fb}^r, Div_{fb}^r = \mathcal{E}_{coh}(\tilde{\mathcal{C}}^i, \mathcal{T}_g^i), \mathcal{E}_{div}(\mathcal{T}_g^i)$

$out = \mathcal{A}_{plan}(Coh_{fb}, Coh_{fb}^r, Div_{fb}, Div_{fb}^r)$

if $out.is_improved$ **is true** **then**

$\mathcal{C} = \mathcal{C}^i$ and $\tilde{\mathcal{C}} = \tilde{\mathcal{C}}^i$ and $\mathcal{T}_g = \mathcal{T}_g^i$

else

$tolerance += 1$

end if

end for

from $\mathcal{E}_{div}$ is incorporated into the topic generation prompt. Once the merging operation is completed for all identified topic groups, $\mathcal{C}$ is updated accordingly by replacing the merged topics with the newly formed topics.

Planning Agent. The refined topics are re-evaluated by $\mathcal{E}_{coh}$ and $\mathcal{E}_{div}$, and the results are then passed to the planning agent ($\mathcal{A}_{plan}$), which determines whether improvements have been achieved. Specifically, the planning agent first invokes LLM to generate the summary reports for both the pre- and post-refinement iteration. It then compares these reports to determine whether any improvement has been achieved. If an improvement is observed, the topic modeling results ($\mathcal{C}$, $\tilde{\mathcal{C}}$, and $\mathcal{T}_g$) are updated with the refined ones and the iteration continues. Otherwise, no updates are made and only the $tolerance$ parameter is incremented by

Table 1. Basic statistics of datasets used for experiments.

DATA SET	DATA SIZE	TOKEN LEN	# OF LABELS
BANKING77	1,947	13	77
BILLS	1,000	233	101
WIKI	1,100	3950	207
CCC	603	116	-
BCR	1,250	15	-
CI	3,000	17	-

1. The refinement loop proceeds until either the maximum tolerance (T) or the maximum number of iterations (e) is reached. Details regarding the prompts used for $\mathcal{A}_{plan}$ and its output format is provided in Appendix B.4.

2.3. Multi-view Topic Parenting

In addition to topics, SELATM also provides parent topics because presenting higher-level topics along with their relationship with base topics can offer valuable insights from an analyst’s perspective. Unlike the topic generation stage, where only text segments are available, parent topics are established based on the topics themselves, allowing additional access to the topic names and descriptions. In this regard, we employ multi-view clustering method (Pattnaik et al., 2024). Specifically, the multi-view embedding of each topic (e_k) is constructed by concatenating the centroid of the text segment embeddings (e_{c_k}), the topic name embedding (e_{t_k}), and the topic description embedding (e_{d_k}). Additionally, the natural language representation of each topic is formed by concatenating the topic name and description. With these representations, we repeat agglomerative clustering to categorize similar topics and use $\mathcal{A}_{TG}$ to generate the name and description of parent topics.

3. Experiment Design

3.1. Datasets

We selected three publicly available datasets, Bills (Hoyle et al., 2022) and Wiki (Gao et al., 2023), which are widely used for topic modeling evaluation, and Banking77 (Casanueva et al., 2020) for its specificity to the finance domain. As noted by Pham et al. 2024, using the entire training corpus for topic modeling is impractical. Therefore, we applied the same sampling strategy as Pham et al., 2024 for our experiments. We also conduct experiments on three in-house business datasets. The Customer Chatbot Conversation (CCC) dataset consists of conversation messages from a consumer banking chatbot application. The Banking Chatbot Review (BCR) dataset contains customer reviews of the banking chatbot application. The Customer Issues (CI) dataset comprises short summaries describing issues raised by customers during their daily banking activities. All three industry datasets does not contain ground-truth labels. Table 1 shows basic statistics of the datasets.

Table 2. Textual labels and corresponding scores used for LLM-as-a-Judge evaluation.

$\mathcal{TA}$	$\mathcal{TC}$	SCORE
INCORRECT	NOT COVERED	0
PARTIALLY CORRECT	MINORLY COVERED	1/3
MOSTLY CORRECT	MOSTLY COVERED	2/3
COMPLETELY CORRECT	COMPLETE	1

3.2. Evaluation Metrics

The public datasets include ground-truth labels, allowing the use of standard clustering-based evaluation metrics as employed in previous studies (Hoyle et al., 2022; Pham et al., 2024). Accordingly, we calculated the Harmonic Mean of Purity (P1), Adjusted Rand Index (ARI), and Normalized Mutual Information (NMI) metrics, with detailed explanations provided in Appendix A.1. These metrics require a single prediction to be compared with the labels. Therefore, for SELATM, we used the topic with the highest contribution as the prediction for evaluation. If there was a tie in top contributions, we treated it as a new topic.

Clustering-based metrics are limited in that they cannot be applied to datasets without ground-truth labels and cannot assess the quality of assigned topic names. Therefore, we evaluated topic accuracy ($\mathcal{TA}$) and topic completeness ($\mathcal{TC}$) (Moon et al., 2026), which measure how accurately topic names are assigned to documents and whether any topics are missing from the documents, respectively. We employed LLM-as-a-Judge building on its recent success on automated evaluations (Zheng et al., 2023; Li et al., 2024; Jang & Silavong, 2025). As LLM are prone to producing inconsistent predictions (Jang & Lukasiewicz, 2023; Wang et al., 2024), we used self-consistency decoding for more reliable evaluations (Wang et al., 2023). Specifically, for each $\mathcal{TA}$ and $\mathcal{TC}$, we prompted LLM to generate five predictions—each with a distinct reasoning path—given a document and its assigned topic name, following the label schema presented in Table 2. The predictions are converted to their corresponding scores, and the final score is computed as the average of these five scores. The prompts of $\mathcal{TA}$ and $\mathcal{TC}$ evaluations are presented in Appendix B.5. When the output consists of multiple topics, we concatenated the generated topics into a single text for measuring $\mathcal{TC}$, considering the definition of the metric. However, this strategy can provide additional benefits when measuring $\mathcal{TA}$. Since SELATM provides a distribution of topics, we used the topic ratios as weights and calculated the weighted average of each individual topic’s $\mathcal{TA}$ score. For methods that do not provide topic weights, we applied the same approach as used for $\mathcal{TC}$.

3.3. Baseline Methods

As baselines, we selected the following approaches from three distinct topic modeling families, based on their re-

ported performance and code availability.

- **Embedded topic models:** BERTopic (Grootendorst, 2022), CTop2Vec (Angelov & Inkpen, 2024).
- **LLM-driven topic generation and assignment:** TopicGPT (Pham et al., 2024), LLoom (Lam et al., 2024), TIDE (Moon et al., 2026).
- **LLM-assisted neural topic models:** LLM-ITL (Yang et al., 2025).

Further details about the baselines and their implementations are provided in Appendix A.2.

3.4. Models and Hyperparameters

For experiments, we used OpenAI models; gpt-4o-2024-08-06 for language generation and text-embedding-3-small-1 for generating embedding vectors. The same LLMs were also applied to baselines unless their code implementations require specific models. Regarding LLM-as-a-Judge, we used a different model, gpt-4.1-mini-2025-04-14.

The number of topics (K) is a necessary hyperparameter for SELATM, TIDE, and LLM-ITL. Based on the number of ground-truth labels, we set K to 100 for the Banking77 and Bills datasets, and to 250 for the Wiki dataset. For the business datasets, which do not contain ground-truth labels, we set K to 50. SELATM requires an additional hyperparameter, n_S , which specifies the number of sampled text segments for each cluster. For each dataset, we identified the best n_S through hyperparameter search; detailed experimental results are provided in Appendix A.3.

4. Experimental Results

In our experiments, we investigated different SELATM variations by setting the number of iterations for the feedback refinement loop to 0, 1, 3, and 5.

4.1. Public Dataset Results

Effect of Segment-level Topic Modeling. The results of the public dataset experiments are presented in Table 3. The results show that SELATM outperforms the baselines on the majority of evaluation metrics. Even without the feedback refinement loop, SELATM achieves the best performance in 10 out of 15 evaluation cases, with 9 of these differences being statistically significant. The improvements are more striking in LLM-as-a-judge metrics, especially for $\mathcal{TA}$, where SELATM outperforms the best-performing baselines by an average of 21%. This finding supports our intuition that generating topics for univocal text clusters reduces task complexity and leads to higher topic modeling accuracy. Moreover, in cases where the baseline models achieve the highest ranking, SELATM mostly

Table 3. Experimental results on the public datasets. We report an average of 5 repetitions. The best performance for each evaluation metric is highlighted in bold; underlined values indicate statistically significant performance gap over the best-performing baselines (or over SELATM when a baseline performs best), determined by t-test with a p-value < 0.05 . ‘SELATM Iter:0’ refers to SELATM results without the refinement loop; italicized values indicate where ‘SELATM Iter:0’ outperforms the baselines.

Models	Banking77					Bills					Wiki				
	P1	ARI	NMI	$\mathcal{TA}$	$\mathcal{TC}$	P1	ARI	NMI	$\mathcal{TA}$	$\mathcal{TC}$	P1	ARI	NMI	$\mathcal{TA}$	$\mathcal{TC}$
BERTopic	.705	.468	.817	.634	.667	.359	.230	.558	.099	.066	.303	.144	.652	.137	.076
C-Top2vec	.557	.406	.788	.470	.704	.424	.230	.767	.350	.379	.490	.341	.685	.501	.574
TopicGPT	.172	.078	.532	.734	.683	.385	.244	.709	.748	.740	.392	.183	.779	.826	.627
LLoM	.111	.037	.322	.283	.259	.159	.048	.295	.501	.424	.110	.046	.424	.544	.353
TIDE	.624	.486	.800	.782	.867	.440	.285	.711	.759	.719	.389	.259	.816	.641	.621
LLM-ITL	.244	.001	.562	.379	.419	.395	.033	.723	.129	.110	.491	.046	.802	.168	.010
SELATM Iter:0	.684	.542	.832	.955	.953	.440	.255	.663	.795	.769	.522	.357	.805	.832	.574
SELATM Iter:1	.680	.539	.832	.962	.956	.440	.260	.661	.803	.782	.524	.359	.803	.807	.567
SELATM Iter:3	.672	.533	.830	.966	.960	.443	.258	.660	.823	.792	.522	.355	.803	.842	.587
SELATM Iter:5	.663	.526	.826	.962	.961	.441	.261	.659	.815	.789	.514	.354	.805	.841	.588

secures the second-best performance—for example, in P1 for Banking77, ARI for Bills, and NMI and $\mathcal{TC}$ for the Wiki dataset. Notably, SELATM demonstrates a substantial margin over the third-ranked model in P1 for Banking77 and ARI for Bills. Although some baselines, such as TIDE and C-Top2vec, achieve the highest scores in certain evaluation cases, it is worth to mention that none of the baselines consistently outperform the others across majority of evaluation cases, as SELATM does. The results highlight the advantage of segment-level topic modeling over the baselines, as it not only offers greater interpretability by providing multiple topics along with their contribution scores, but also outperforms the baselines across various evaluation metrics. Several output examples of the public datasets are provided in Appendix C.1.

Effect of the Refinement Loop. Regarding the conventional clustering-based metrics, applying the feedback refinement loop does not result in significant performance differences. Specifically, no statistically significant differences are observed in P1, ARI, and NMI metrics for the Bills and Wiki datasets using a t-test under a p-value < 0.05 . In the Banking77 dataset, there is a trend that the clustering-based metrics slightly decrease with more iterations, where the differences in P1 and ARI become statistically significant after 5 iterations. However, these are expected outcomes, as each iteration involves split and merge operations on only a small number of problematic topics. Additionally, the refinement process not only performs split and merge operations but also focuses on renaming topics—a factor that cannot be captured by clustering-based metrics. Consequently, most clusters remain unchanged, leading to minimal impact on clustering-based metrics.

On the contrary, the LLM-as-a-Judge metrics show significant improvement, with the best performance achieved mostly with 3 refinement iterations. Both $\mathcal{TA}$ and $\mathcal{TC}$ exhibit statistically significant gains, as determined by a t-test with p-value < 0.05 , across all three datasets when using 3

Table 4. Experimental results on the business datasets. We report an average of 5 repetitions. The best performance for each evaluation metric is highlighted in bold; underlined values indicate statistically significant performance gap over the best-performing baselines (or over SELATM when a baseline performs best), determined by a t-test with a p-value < 0.05 .

Models	CCC		BCR		CI	
	$\mathcal{TA}$	$\mathcal{TC}$	$\mathcal{TA}$	$\mathcal{TC}$	$\mathcal{TA}$	$\mathcal{TC}$
C-Top2vec	.723	.886	.050	.016	.607	.829
TIDE	.750	.773	.502	.555	.836	.842
LLM-ITL	.453	.409	.344	.369	.228	.189
SELATM Iter:0	.825	.959	.832	.870	.845	.780
SELATM Iter:1	.840	.957	.845	.880	.847	.782
SELATM Iter:3	.849	.962	.851	.884	.839	.773
SELATM Iter:5	.785	.896	.870	.899	.784	.722

iterations compared to results without the feedback refinement loop. Additionally, an interesting result is observed for the LLM-ITL approach, which achieved relatively high P1 and NMI scores, as reported in its original paper. However, the $\mathcal{TA}$ and $\mathcal{TC}$ scores for LLM-ITL are significantly lower than those of TIDE, TopicGPT, and SELATM.¹ This suggests that, while a word-level LLM analysis method like LLM-ITL offers the advantage of reduced computational overhead, naming topics solely based on top-related words has the potential drawback of producing less accurate and less comprehensive topic names.

4.2. Business Dataset Results

For this experiment, we selected C-Top2vec, TIDE, and LLM-ITL as baselines. C-Top2vec and TIDE were chosen because they outperformed others within their respective topic modeling families by a large margin, while LLM-ITL was included as it is the sole representative of the LLM-assisted neural topic model family. As the business datasets do not contain labels, only LLM-as-a-Judge metrics are

¹Also, the ARI score of LLM-ITL is considerably low.

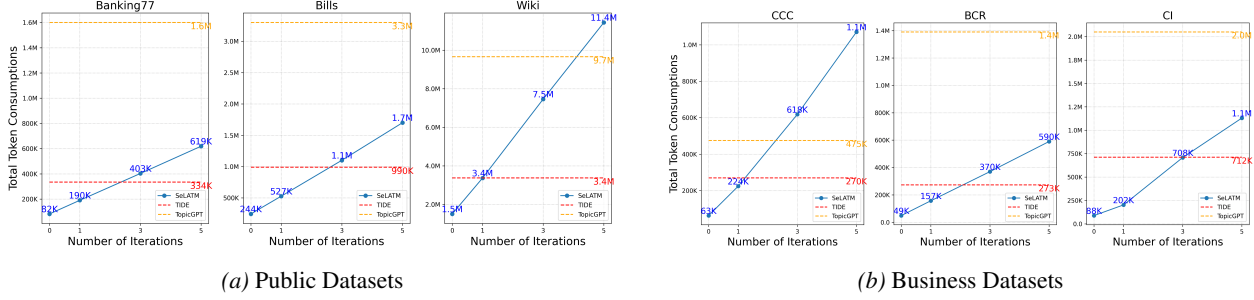


Figure 1. An average LLM token consumption comparison between SELATM, TIDE, and TopicGPT.

reported. The results are presented in Table 4. Several output examples of the business datasets are provided in Appendix C.2.

Overall, the results exhibit a similar trend to those observed in the public dataset experiments. Even without the refinement loop, SELATM statistically significantly outperforms baseline approaches in 5 out of 6 evaluation cases (based on a t-test with p -value < 0.05), with the only exception being $\mathcal{TC}$ in the CI dataset. Additionally, SELATM demonstrates consistently strong performance across all evaluation cases, in contrast to TIDE, which underperforms on the BCR dataset, and C-Top2vec, which performs poorly on $\mathcal{TA}$ for the CI dataset and fails entirely on the BCR dataset.² The performance gap between baselines and SELATM becomes even more pronounced with the feedback refinement. Specifically, the best results are achieved with 3 iterations for the CCC dataset and 5 iterations for the BCR dataset, with improvements that are statistically significant compared to SELATM without the refinement loop, as determined by a t-test with p -value < 0.05 (except for $\mathcal{TC}$ of the CCC dataset).

Although our proposed approach demonstrates notable improvements, we observed some limitations. First, the feedback refinement loop does not yield statistically significant improvement in the CI dataset. Additionally, we observed performance degradations after 5 iterations in both the CCC and CI datasets, which is quite similar to the results seen with the Banking77 dataset. Through qualitative analysis, we found that the split operation generally stops after 3 iterations, while the merge operation continues, inevitably resulting in larger but more ambiguous and generic topics, thereby leading to a decrease in the evaluation metrics. To address this issue, efforts should be made to adjust the judgment bias of $\mathcal{A}_{plan}$ so that excessive merging is recognized as a sign of negative change; we leave this as an area for future work. Despite these limitations, results from both public and business datasets indicate that SELATM is likely to produce higher-quality topics than the baseline approaches, while also offering the benefit of greater inter-

pretability through the quantification of contributions from multi-topics.

4.3. LLM-Token Consumption Comparison

As noted above, a key practical advantage of SELATM over LLM-driven topic modeling approaches that use topic generation followed by assignment is its LLM resource efficiency. To verify this, we measured the total LLM token consumption of the entire SELATM process and compared them with that of TIDE and TopicGPT. LLoM is excluded from this comparison because its performance, as shown in Table 3, is significantly lower than that of the other methods in the same family. Figure 1 illustrates an average token consumption comparison. Without the feedback refinement loop, SELATM uses significantly fewer LLM resources—averaging only 24.4% and 8.19% of TIDE’s and TopicGPT’s token consumption, respectively. With the feedback refinement loop, SELATM still uses only 55% of TIDE’s token consumption on average with one iteration, but begins to exceed TIDE’s consumption after three iterations. Compared to TopicGPT, SELATM uses fewer tokens even after 3 - 5 iterations. Given that SELATM already outperforms TIDE and TopicGPT with zero or one iteration of the feedback refinement loop, as shown in Tables 3 and 4, the figures in Figure 1 highlight the superior resource efficiency of SELATM.

4.4. Parent Topic Modeling Performance

To validate the quality of the parent topics, we measured LLM-as-a-Judge evaluation metrics, as most of the datasets do not contain higher-level topic labels. We compared the performance of our proposed method with TIDE, which is the best-performing baseline capable of providing parent topics. The results are illustrated in Figure 2.

The results indicate that the parent topic achieves 92% of the base topic’s performance on average, which is notably high given that parent topics encompass a much broader range of content. Moreover, SELATM produces significantly better parent topic performance compared to TIDE. This improvement is especially pronounced in the Bills and Wiki datasets,

²We observed that C-Top2vec generates only two topics on the BCR dataset, even with various hyperparameter settings.

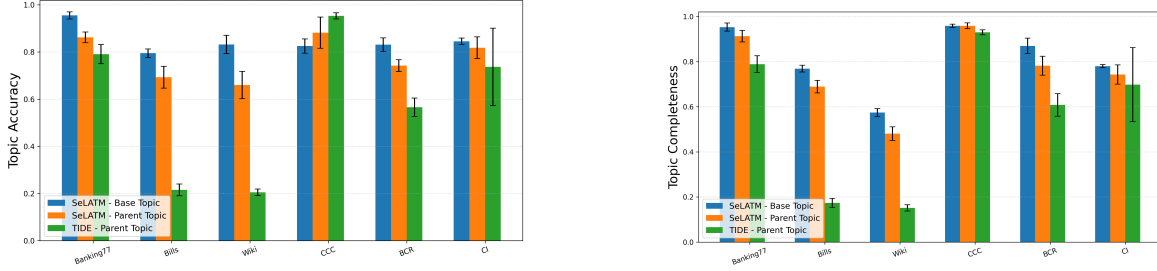


Figure 2. The $\mathcal{TA}$ and $\mathcal{TC}$ results on the parent topics.

where SELATM achieves $\mathcal{TA}$ and $\mathcal{TC}$ scores that are 2.2 and 2.6 times higher, respectively, than those of TIDE. For the other datasets, SELATM improves performance by 11% in $\mathcal{TA}$ and 14% in $\mathcal{TC}$ compared to TIDE, with the only exception being the CCC dataset. TIDE achieves a higher $\mathcal{TA}$ in the CCC dataset; however, the difference is not statistically significant. The results highlight the advantage of multi-view clustering over fully relying on LLM for identifying topic hierarchies.

Additionally, we conducted several ablation studies, which are presented in Appendix A.4 due to the page limit constraint. We highly encourage readers to examine the ablation studies to investigate the benefits of using agglomerative clustering and the split operation.

5. Related Works

Topic modeling is an NLP technique to automatically discover hidden semantic patterns underlying in a text corpus. Traditional statistical methods, such as Latent Dirichlet Allocation (LDA) (Blei et al., 2003) and Non-negative Matrix Factorization (NMF) (Févotte & Idier, 2011), assume that topics are latent variables between words and documents, and attempt to uncover document-topic and topic-word distributions. A significant downside of these methods is that they rely on word-count based bag-of-words representations, which overlook the semantic meanings and relationships between words. In an attempt to overcome this drawback, embedded topic modeling approaches that utilize dense vector representations have been proposed (Sia et al., 2020; Angelov, 2020; Grootendorst, 2022; Angelov & Inkpen, 2024). The recent success of LLMs has enabled researchers to apply the models for topic modeling by directly asking LLMs to identify topics underlying a text corpus and assign them to each individual document (Mu et al., 2024; Doi et al., 2024; Pham et al., 2024; Lam et al., 2024; Liu et al., 2025). A distinctive feature of LLM-based methods is their capability to generate interpretable and human-readable topics, unlike the aforementioned approaches where topics are represented as a set of related words. However, these document-level LLM analysis approaches have the drawback of significant computational overhead. To address this

issue, LLM-ITL (Yang et al., 2025) utilizes conventional neural topic modeling for document analysis and employs LLMs primarily for refinement, thereby reducing computational overhead.

6. Conclusion

The advent and rapid progress of LLMs are propelling the field of topic modeling forward. Recent approaches actively employ LLMs to identify underlying topics within a corpus and assign them to individual documents. However, despite their promising results, recent LLM-based topic modeling approaches have several drawbacks: (1) inability to quantify the contribution of multiple topics, (2) high computational demands due to document-level analysis, and (3) inaccurate topic naming resulting from insufficient refinement processes. In this paper, we propose SELATM to address these limits, which is characterized by segment-level topic modeling and agentic feedback refinement loop. By assigning topics at the text segment level, SELATM enables documents to be represented as distributions over multiple topics, thereby quantifying the contribution of each topic and elucidating the association between specific segments and their corresponding topics. Also, SELATM eliminates the need for a separate topic assignment process, resulting in a substantial reduction in LLM computational expense. Furthermore, both the segment-level topic modeling and feedback refinement loop facilitate the generation of more precise topics by reducing task complexity and enabling collaborative refinement among diverse LLM agents. Extensive experiments on public and business datasets demonstrate that SELATM successfully addresses these challenges by enhancing interpretability through topic contribution quantification, reducing LLM computation overhead, and producing better-quality topics.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

References

- Angelov, D. Top2vec: Distributed representations of topics. *arXiv preprint arXiv:2008.09470*, 2020.
- Angelov, D. and Inkpen, D. Topic modeling: Contextual token embeddings are all you need. In Al-Onaizan, Y., Bansal, M., and Chen, Y.-N. (eds.), *Findings of the Association for Computational Linguistics: EMNLP 2024*, pp. 13528–13539, Miami, Florida, USA, November 2024. Association for Computational Linguistics.
- Asmussen, C. B. and Møller, C. Smart literature review: a practical topic modelling approach to exploratory literature review. *Journal of Big Data*, 6(1):1–18, 2019.
- Blei, D. M., Ng, A. Y., and Jordan, M. I. Latent dirichlet allocation. *Journal of machine Learning research*, 3(Jan): 993–1022, 2003.
- Carbonell, J. and Goldstein, J. The use of MMR, diversity-based reranking for reordering documents and producing summaries. In *Proceedings of the 21st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR '98*, pp. 335–336, New York, NY, USA, 1998. Association for Computing Machinery. ISBN 1581130155.
- Casanueva, I., Temcinas, T., Gerz, D., Henderson, M., and Vulic, I. Efficient intent detection with dual sentence encoders. In *Proceedings of the 2nd Workshop on NLP for ConvAI - ACL 2020*, mar 2020. Data available at <https://github.com/PolyAI-LDN/task-specific-datasets>.
- Chalkidis, I., Jana, A., Hartung, D., Bommarito, M., Androutsopoulos, I., Katz, D., and Aletras, N. LexGLUE: A benchmark dataset for legal language understanding in English. In Muresan, S., Nakov, P., and Villavicencio, A. (eds.), *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 4310–4330, Dublin, Ireland, May 2022. Association for Computational Linguistics.
- Dagan, G., Keller, F., and Lascarides, A. Dynamic planning with a llm. In *Language Gamification-NeurIPS 2024 Workshop*, 2024.
- Doi, T., Isonuma, M., and Yanaka, H. Topic modeling for short texts with large language models. In Fu, X. and Fleisig, E. (eds.), *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 4: Student Research Workshop)*, pp. 21–33, Bangkok, Thailand, August 2024. Association for Computational Linguistics. ISBN 979-8-89176-097-4.
- Duarte, A. V., Marques, J. D., Graça, M., Freire, M., Li, L., and Oliveira, A. L. LumberChunker: Long-Form Narrative Document Segmentation. In Al-Onaizan, Y., Bansal, M., and Chen, Y.-N. (eds.), *Findings of the Association for Computational Linguistics: EMNLP 2024*, pp. 6473–6486, Miami, Florida, USA, November 2024. Association for Computational Linguistics.
- Févotte, C. and Idier, J. Algorithms for nonnegative matrix factorization with the β -divergence. *Neural computation*, 23(9):2421–2456, 2011.
- Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Guo, Q., Wang, M., and Wang, H. Retrieval-Augmented Generation for Large Language Models: A Survey. *CoRR*, abs/2312.10997, 2023.
- Grootendorst, M. Bertopic: Neural topic modeling with a class-based tf-idf procedure. *arXiv preprint arXiv:2203.05794*, 2022.
- Halliday, M. A. K. and Matthiessen, C. M. *Halliday's introduction to functional grammar*. Routledge, 4 edition, 2013.
- Hoyle, A. M., Sarkar, R., Goel, P., and Resnik, P. Are Neural Topic Models Broken? In Goldberg, Y., Kozareva, Z., and Zhang, Y. (eds.), *Findings of the Association for Computational Linguistics: EMNLP 2022*, pp. 5321–5344, Abu Dhabi, United Arab Emirates, December 2022. Association for Computational Linguistics.
- Hu, S., Yu, D., Zhang, Y., Yang, Z., Zhou, L., Jiang, X., Li, Y., Fei, B., and Liu, J. Finlbench: A benchmark for evaluating large language models on long-text financial documents. In *International Conference on Neural Information Processing*, pp. 244–258. Springer, 2025.
- Hubert, L. and Arabie, P. Comparing partitions. *Journal of classification*, 2:193–218, 1985.
- Jang, M. E. and Lukasiewicz, T. Consistency analysis of ChatGPT. In Bouamor, H., Pino, J., and Bali, K. (eds.), *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 15970–15985, Singapore, December 2023. Association for Computational Linguistics.
- Jang, M. E. and Silavong, F. InstaJudge: Aligning judgment bias of LLM-as-judge with humans in industry applications. In Potdar, S., Rojas-Barahona, L., and Montella, S. (eds.), *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pp. 1158–1172, Suzhou (China), November 2025. Association for Computational Linguistics. ISBN 979-8-89176-333-3.
- Jang, M. E. and Stikkel, G. Leveraging natural language processing and large language models for assisting due diligence in the legal domain. In Yang, Y., Davani, A., Sil, A., and Kumar, A. (eds.), *Proceedings of the 2024*

- Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 6: Industry Track)*, pp. 155–164, Mexico City, Mexico, June 2024. Association for Computational Linguistics.
- Kambhampati, S. Can large language models reason and plan? *Annals of the New York Academy of Sciences*, 1534 (1):15–18, 2024.
- Lam, M. S., Teoh, J., Landay, J. A., Heer, J., and Bernstein, M. S. Concept induction: Analyzing unstructured text with high-level concepts using Iloom. In *Proceedings of the CHI Conference on Human Factors in Computing Systems*, pp. 1–28, 2024.
- Li, Z., Xu, X., Shen, T., Xu, C., Gu, J.-C., Lai, Y., Tao, C., and Ma, S. Leveraging large language models for NLG evaluation: Advances and challenges. In Al-Onaizan, Y., Bansal, M., and Chen, Y.-N. (eds.), *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 16028–16045, Miami, Florida, USA, November 2024. Association for Computational Linguistics.
- Liu, J., Shang, Z., Ke, W., Wang, P., Luo, Z., Liu, J., Li, G., and Li, Y. LLM-guided semantic-aware clustering for topic modeling. In Che, W., Nabende, J., Shutova, E., and Pilehvar, M. T. (eds.), *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 18420–18435, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-251-0.
- Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., and Liang, P. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.
- Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegrefe, S., Alon, U., Dziri, N., Prabhunoye, S., Yang, Y., et al. Self-refine: Iterative refinement with self-feedback. *Advances in Neural Information Processing Systems*, 36:46534–46594, 2023.
- Masry, A. and Hajian, A. Longfin: A multimodal document understanding model for long financial domain documents. *arXiv preprint arXiv:2401.15050*, 2024.
- Miao, Y., Grefenstette, E., and Blunsom, P. Discovering discrete latent topics with neural variational inference. In *International conference on machine learning*, pp. 2410–2419. PMLR, 2017.
- Moens, M.-F. Using patterns of thematic progression for building a table of contents of a text. *Natural Language Engineering*, 14(2):145–172, 2008.
- Moon, S. Y., Jang, M. E., Luo, H., Xiao, C., Georgiadis, A., and Silavong, F. Industry-aligned granular topic modeling, 2026.
- Mu, Y., Dong, C., Bontcheva, K., and Song, X. Large language models offer an alternative to the traditional approach of topic modelling. In Calzolari, N., Kan, M.-Y., Hoste, V., Lenci, A., Sakti, S., and Xue, N. (eds.), *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, pp. 10160–10171, Torino, Italia, May 2024. ELRA and ICCL.
- Murtagh, F. and Legendre, P. Ward’s hierarchical agglomerative clustering method: which algorithms implement ward’s criterion? *Journal of classification*, 31(3):274–295, 2014.
- Pattanaik, A., George, C., Tripathi, R. K., Vutla, S., and Vepa, J. Improving Hierarchical Text Clustering with LLM-guided Multi-view Cluster Representation. In Dernoncourt, F., Preotiu-Pietro, D., and Shimorina, A. (eds.), *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pp. 719–727, Miami, Florida, US, November 2024. Association for Computational Linguistics.
- Pham, C., Hoyle, A., Sun, S., Resnik, P., and Iyyer, M. TopicGPT: A prompt-based topic modeling framework. In Duh, K., Gomez, H., and Bethard, S. (eds.), *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pp. 2956–2984, Mexico City, Mexico, June 2024. Association for Computational Linguistics.
- Ranaei, S., Suominen, A., and Dedehayir, O. A topic model analysis of science and technology linkages: A case study in pharmaceutical industry. In *2017 IEEE Technology & Engineering Management Conference (TEMSCON)*, pp. 49–54. IEEE, 2017.
- Rand, W. M. Objective criteria for the evaluation of clustering methods. *Journal of the American Statistical association*, 66(336):846–850, 1971.
- Ravi, R., Bradshaw, D., Ruberto, S., Jahangirova, G., and Terragni, V. Llmloop: Improving llm-generated code and tests through automated iterative feedback loops. *ICSME. IEEE*, 2025.
- Shannon, C. E. A mathematical theory of communication. *The Bell system technical journal*, 27(3):379–423, 1948.
- Sia, S., Dalmia, A., and Mielke, S. J. Tired of topic models? clusters of pretrained word embeddings make for fast and good topics too! In Webber, B., Cohn, T., He, Y., and Liu,

- Y. (eds.), *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 1728–1736, Online, November 2020. Association for Computational Linguistics.
- Song, C. H., Wu, J., Washington, C., Sadler, B. M., Chao, W.-L., and Su, Y. Llm-planner: Few-shot grounded planning for embodied agents with large language models. In *Proceedings of the IEEE/CVF international conference on computer vision*, pp. 2998–3009, 2023.
- Steinley, D. Properties of the hubert-arable adjusted rand index. *Psychological methods*, 9(3):386, 2004.
- Strehl, A. and Ghosh, J. Cluster ensembles—a knowledge reuse framework for combining multiple partitions. *Journal of machine learning research*, 3(Dec):583–617, 2002.
- Strunk Jr, W. and White, E. B. *The elements of style illustrated*. Penguin, 2007.
- Sypherd, C. and Belle, V. Practical considerations for agentic llm systems. *CoRR*, 2024.
- Valmeekam, K., Olmo, A., Sreedharan, S., and Kambhampati, S. Large language models still can’t plan (a benchmark for llms on planning and reasoning about change). In *NeurIPS 2022 Foundation Models for Decision Making Workshop*, 2022.
- Valmeekam, K., Marquez, M., Sreedharan, S., and Kambhampati, S. On the planning abilities of large language models—a critical investigation. *Advances in Neural Information Processing Systems*, 36:75993–76005, 2023.
- Vinh, N. X., Epps, J., and Bailey, J. Information theoretic measures for clusterings comparison: is a correction for chance necessary? In *Proceedings of the 26th Annual International Conference on Machine Learning, ICML ’09*, pp. 1073–1080, New York, NY, USA, 2009. Association for Computing Machinery. ISBN 9781605585161.
- Wang, P., Li, L., Chen, L., Cai, Z., Zhu, D., Lin, B., Cao, Y., Kong, L., Liu, Q., Liu, T., and Sui, Z. Large language models are not fair evaluators. In Ku, L.-W., Martins, A., and Srikumar, V. (eds.), *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 9440–9450, Bangkok, Thailand, August 2024. Association for Computational Linguistics.
- Wang, X., Wei, J., Schuurmans, D., Le, Q. V., Chi, E. H., Narang, S., Chowdhery, A., and Zhou, D. Self-consistency improves chain of thought reasoning in language models. In *The Eleventh International Conference on Learning Representations*, 2023.
- Xiong, H., Cheng, Y., Zhao, W., and Liu, J. Analyzing scientific research topics in manufacturing field using a topic model. *Computers & Industrial Engineering*, 135: 333–347, 2019.
- Yang, X., Zhao, H., Xu, W., Qi, Y., Lu, J., Phung, D., and Du, L. Neural topic modeling with large language models in the loop. In Che, W., Nabende, J., Shutova, E., and Pilehvar, M. T. (eds.), *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 1377–1401, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-251-0.
- Yin, L. and Wang, Z. Llm-autodiff: Auto-differentiate any llm workflow. *arXiv preprint arXiv:2501.16673*, 2025.
- Yuksekgonul, M., Bianchi, F., Boen, J., Liu, S., Huang, Z., Guestrin, C., and Zou, J. Textgrad: Automatic” differentiation” via text. *arXiv preprint arXiv:2406.07496*, 2024.
- Yuksel, K. A., Castro Ferreira, T., Al-Badrashiny, M., and Sawaf, H. A multi-AI agent system for autonomous optimization of agentic AI solutions via iterative refinement and LLM-driven feedback loops. In Kamaloo, E., Gontier, N., Lu, X. H., Dziri, N., Murty, S., and Lacoste, A. (eds.), *Proceedings of the 1st Workshop for Research on Agent Language Models (REALM 2025)*, pp. 52–62, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-264-0.
- Zhao, Y. *Criterion Functions for Document Clustering*. PhD thesis, University of Minnesota, USA, 2005. AAI3180039.
- Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E., Zhang, H., Gonzalez, J. E., and Stoica, I. Judging llm-as-a-judge with mt-bench and chatbot arena. In Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., and Levine, S. (eds.), *Advances in Neural Information Processing Systems*, volume 36, pp. 46595–46623. Curran Associates, Inc., 2023.
- Zhou, Y., Dai, S., Cao, Z., Zhang, X., and Xu, J. Length-induced embedding collapse in transformer-based models. *Openreview*, 2024.

A. Appendix

A.1. Clustering-based Evaluation Metrics

Regarding the public datasets, we used clustering-based evaluation metrics in accordance with previous studies. Let us assume that there are N instances and two clusters $U = \{u_1, u_2, \dots, u_K\}$ and $V = \{v_1, v_2, \dots, v_C\}$, where u_i and v_i represent the i -th cluster assigned to U and V , respectively.

Harmonic Mean of Purity (P1). Purity is an intuitive metric that is analogous to precision (Zhao, 2005). Specifically, it evaluates how many ground-truth labels are contained within all documents assigned to a single predicted cluster. A small number of ground-truth labels in a single predicted cluster intuitively suggests a high degree of correspondence between the identified and ground-truth concepts (Hoyle et al., 2022). Provided V to be considered as the ground-truth, purity is computed as follows:

$$P(U, V) = \frac{1}{N} \sum_k \max_c |C_{U_k} \cap T_{V_c}|,$$

where C_{U_k} denotes the set of instances assigned to cluster U_k , and T_{V_c} refers to the set of instances in ground-truth label V_c . However, purity is an unsymmetrical metric; therefore, the Harmonic Mean of Purity (P1) is more commonly used in practice, which is calculated as follows:

$$P1(U, V) = \frac{2 \times P(U, V) \times P(V, U)}{P(U, V) + P(V, U)}.$$

Adjusted Random Index (ARI). Random Index (RI, Rand 1971) is a metric that evaluates whether two clustering results agree or disagree, which is calculated by:

$$RI(U, V) = (N_{00} + N_{11}) / \binom{N}{2},$$

where N_{00} and N_{11} denote the number of pairs that are assigned to different clusters and same clusters, respectively, in both U and V . However, the RI metric has certain limitations; for instance, although its theoretical range is from 0 to 1, its practical values often lie within the narrow range of 0.5 to 1 (Vinh et al., 2009). To address this, the ARI is more commonly used in practice, as it adjusts the RI to account for the change groupings of elements (Steinley, 2004). The ARI is calculated as follows:

$$ARI(U, V) = \frac{RI - RI_E}{\max(RI) - RI_E},$$

where RI_E represents the expected value of the RI for random clusters, and $\max(RI)$ denotes the maximum possible value of the RI. The ARI ranges from -1 to 1, with 1 indicating perfect agreement between the two clusters, 0

corresponding to random cluster assignments, and negative values reflecting less agreement than would be expected by chance (Hubert & Arabie, 1985; Vinh et al., 2009).

Normalized Mutual Information (NMI). NMI is a metric used to evaluate the similarity between two clustering results, under the concept of mutual information (MI, Shannon 1948), which quantifies the amount of information shared between two random variables. As the name suggests, the NMI normalizes the MI to a value between 0 and 1, thereby reducing MI’s sensitivity to variations in the number of clusters (Strehl & Ghosh, 2002). The NMI is calculated as follows:

$$NMI(U, V) = \frac{2 \times \mathcal{I}(U, V)}{\mathcal{H}_U + \mathcal{H}_V},$$

where $\mathcal{I}(U, V)$ denotes the MI between the clusters U and V . $\mathcal{H}_U$ and $\mathcal{H}_V$ refer to the entropies of the clusters U and V , respectively.

A.2. Baseline Models and Implementation Details

BERTopic (Grootendorst, 2022): BERTopic belongs to an embedded topic models family that employs document embeddings derived from a pre-trained language model (PLM). The embeddings are then clustered, and for each cluster, topic representations are generated using a class-based TF-IDF method. For implementation, we used the public BERTopic library.³ In accordance with the paper, we employed `all-mpnet-base-v2` as a backbone embedding model and used the *fit* API with its default configurations.

C-Top2Vec (Angelov & Inkpen, 2024): This method is more improved embedded topic model compared to BERTopic. It segments a document using a sliding window and represents it within multidimensional embedding vector representations. The vector representations are clustered, with topic vectors defined as the centroids of these clusters. Document-topic distributions are then determined according to the topics to which the segments are assigned. Finally, each topic is labeled using words and phrases that are proximal to the corresponding topic vectors. For implementation, we used the original code shared by the authors.⁴ Similar to the paper, we employed `all-mpnet-base-v2` as an embedding model and applied the same hyperparameter settings as the authors, which are demonstrated in Table 8 of their paper (Angelov & Inkpen, 2024).

TopicGPT (Pham et al., 2024): TopicGPT is an LLM-based approach comprising two stages: topic generation, which identifies the underlying topics within a given corpus, and topic assignment, which allocates these generated topics

³<https://github.com/MaartenGr/BERTopic>

⁴<https://github.com/ddangelov/Top2Vec>

Table 5. Results of n_S search. We report an average of 5 repetitions. For each dataset, the best performance achieved by SELATM with varying n_S values is highlighted in bold. Regarding the Banking77 and BCR datasets, $n_S = 80$ was not evaluated because the largest cluster in each dataset contained fewer than 80 instances. The best baseline performance is underlined if it achieves a higher score than the lowest performance of SELATM across the different n_S values.

n_S	Banking77		Bills		Wiki		CCC		BCR		CI	
	$\mathcal{TA}$	$\mathcal{TC}$	$\mathcal{TA}$	$\mathcal{TC}$	$\mathcal{TA}$	$\mathcal{TC}$	$\mathcal{TA}$	$\mathcal{TC}$	$\mathcal{TA}$	$\mathcal{TC}$	$\mathcal{TA}$	$\mathcal{TC}$
30	.932	.924	.775	.744	.814	.586	.791	.954	.831	.870	.753	.703
40	.935	.927	.795	.769	.832	.574	.810	.942	.809	.844	.807	.735
50	.947	.935	.774	.764	.801	.573	.825	.959	.825	.857	.796	.723
60	.955	.950	.787	.755	.803	.587	.812	.958	.806	.839	.832	.762
70	.955	.953	.777	.734	.805	.584	.821	.958	.805	.828	.845	.780
80	-	-	.781	.737	.824	.603	.826	.958	-	-	.806	.747
Best Baseline Result	.782	.867	.759	.719	.641	<u>.621</u>	.750	.886	.502	.555	<u>.836</u>	<u>.842</u>

to individual documents. For implementation, we used the public code shared by the authors.⁵ For experiments, we used the full pipeline including `refinement`. To make fair comparison with other methods that generate more granular topics, we generated and evaluated on low-level topic produced by `generate_topic_lvl2` API.

LLoom (Lam et al., 2024). This LLM-based framework is designed to identify high-level concepts. The method involves three steps: DISTILL-CLUSTER-SYNTHESIZE steps. The DISTILL step condenses text data through summarization. The CLUSTER step performs clustering and SYNTHESIZE step identifies high-level concepts underlying in each cluster. The DISTILL and SYNTHESIZE steps are LLMs-driven processes. For implementation, we used the official code shared by the authors.⁶ For all of our experiments, we utilized the full pipeline exposed by `gen_auto` API, using the default parameter settings.

LLM-ITL (Yang et al., 2025). This approach utilizes LLMs for topic refinement. Specifically, topics—represented as word distributions—and document representations—expressed as topic distributions—are first learned through neural topic modeling (NTM). Subsequently, an LLM generates the topic name based on the topic word distribution and refines the topic words by removing irrelevant terms and adding new, relevant ones. A distinctive feature of LLM-ITL is its use of Optimal Transport (OT) distance to optimally align the topic word distributions produced by the neural topic model (NTM) with the refined topic word distributions generated by the LLM. The approach has several advantages in that it can be readily applied to various NTM methods and significantly reduces LLM usage overhead. For implementation, we used the original code shared by the authors.⁷ We utilized NVDM (Miao et al., 2017) as the base NTM method and adopted the same

training hyperparameters as those used by the authors, except for the number of topics, which was set to match our experimental setup.

TIDE (Moon et al., 2026). This approach is a modified version of TopicGPT. Unlike TopicGPT, which generates topics from entire documents, TIDE first performs clustering and then generates topics for each cluster, enabling to use fewer LLM calls. We introduced TIDE as a baseline because it enables users to decide the number of topics, allowing a fairer comparison with LLM-ITL and SELATM. Although the code is not publicly available, the paper provides a clear description of the algorithm and detailed information about the prompt designs. Furthermore, the algorithm is straightforward to replicate; therefore, we implemented it ourselves by referring to the paper.

A.3. Hyperparameter Search for Optimal n_S

In SELATM, n_S is a crucial hyperparameter that determines the number of text segment samples for each cluster used for topic generation and, consequently, may influence the quality of the generated topics, e.g., the name and description of topics. Accordingly, we conducted a thorough hyperparameter search across all datasets, varying the value from 30 to 80 in increments of 10. We measured $\mathcal{TA}$ and $\mathcal{TC}$ for the hyperparameter search, as the clustering-based metrics are already determined during segmentation and clustering phase and do not depend on the sampled text segments. The results are summarized in Table 5.

Based on the search result, we set n_S to 70 for the Banking77 and CI datasets, 40 for the Bills and Wiki datasets, 50 for the CCC dataset, and 30 for the BCR dataset. However, the effect of n_S was less pronounced than expected, with only marginal performance differences across most datasets, except for the CI dataset. Moreover, even the lowest performance achieved by SELATM with specific n_S exceeds that of the best baseline approach in 9 out of 12 evaluation cases. Therefore, we recommend setting n_S to around

⁵<https://github.com/chtmp223/topicGPT>

⁶<https://github.com/michelle123lam/loom>

⁷<https://github.com/Xiaohao-Yang/LLM-ITL>

Table 6. Ablation study on the clustering methods. We report an average of 5 repetitions (Avg.) and their standard deviations (Std.). The best average performance for each evaluation metric is highlighted in bold. The best performance exhibits statistically significant gap (*) determined by t-test with a p-value < 0.05.

	Models	Banking77			Bills			Wiki		
		P1	ARI	NMI	P1	ARI	NMI	P1	ARI	NMI
Avg.	K-Means	.664	.524	.819	.415	.247	.680*	.500	.334	.788
	Agglomerative	.684*	.542*	.832*	.440*	.255	.663	.522*	.357*	.805*
Std.	K-Means	.014	.018	.007	.011	.026	.005	.012	.023	.004
	Agglomerative	.002	.002	.000	.005	.002	.001	.004	.005	.001

40–50 if there is insufficient capacity to conduct a thorough hyperparameter search.

A.4. Ablation Study

In this section, we conduct ablation studies to ascertain the benefit of utilizing agglomerative clustering and the `split` operation.

Comparison with K-Means Clustering. A leading cause of adopting agglomerative clustering is because the approach is more stable than K-Means clustering, which tends to have greater variability. As part of our ablation study, we evaluated and compared the performance of K-Means clustering against agglomerative clustering. Specifically, the experiments were conducted on public datasets, and clustering-based evaluation metrics were used to assess performance. No feedback refinement loop is applied. The results are summarized in Table 6.

The findings demonstrate that agglomerative clustering exhibits greater stability and achieves superior evaluation metric scores compared to K-Means. With the exception of the Bills dataset, agglomerative clustering demonstrates statistically significantly superior performance compared to K-Means across all evaluation metrics. While K-Means achieves a marginally higher NMI score on the Bills dataset, agglomerative clustering surpasses it on the other two metrics. In terms of standard deviations, agglomerative clustering exhibits substantially lower variability compared to K-Means, which further support our choice of using agglomerative clustering for this study. M

Effect of Split Operation. One distinctive feature of our proposed method’s refinement process is the inclusion of a `split` operation, whereas most existing LLM-driven topic modeling approaches rely solely on topic merging. In order to assess the impact of the `split` operation, we conducted an ablation study in which $\mathcal{E}_{coh}$ and $\mathcal{A}_{split}$ were removed from the refinement loop, so that only $\mathcal{E}_{div}$ and $\mathcal{A}_{merge}$ were utilized. The experiments were conducted on the public datasets, and LLM-as-a-Judge metrics were measured, since the feedback refinement loop has only a minor impact on clustering-based metrics but a significant effect on LLM-as-

a-Judge metrics. The results are summarized in Table 7.

In the Banking77 dataset, the inclusion of the `split` operation does not result in any significant differences. However, considerable performance degradations are observed in the Bills and Wiki datasets, where the absence of the `split` operation consistently resulted in statistically significantly lower performance across all iterations compared to the full feedback refinement loop. Moreover, in both datasets, performance continues to decline with additional iterations when the `split` operation is omitted, which is an expected outcome since excessive topic merging leads to broader and more ambiguous topics. Additionally, we observed that the `split` operation ceases to be applied after 2 to 3 iterations, which is closely related to why optimal performance is typically achieved within 1 to 3 iterations. These findings substantiate our claim regarding the necessity of including the `split` operation in the refinement process.

B. Prompt Designs

B.1. Topic Generation Prompt

To generate topic names and descriptions of a given text segment cluster, we use the following prompt, where `n_topic_words` denotes a user-defined hyperparameter with a default value of 2. `topic_definition` is an optional prompt that allows users to specify a customized definition of a topic, which is particularly useful when working with real-world industry data.

Prompt for Topic Generation Agent

You will receive a list of text segments that belong to the same cluster. Each text segment is identified by a unique ID. Given the segments, your task is to read the contents, then generate a topic that can describe as many text segments from this group as possible.

Along with the topic, you need to generate a short description of the meaning of the topic given by yourself.

[Instructions]

Table 7. Ablation study on the `split` operation. We report an average of 5 repetitions. The performance of ‘`split+merge`’ is underlined when it is statistically significantly higher than that of ‘`w/o split`’ in the corresponding iteration under a t-test with a p-value < 0.05 .

	Iterations	Banking77		Bills		Wiki	
		<i>TA</i>	<i>TC</i>	<i>TA</i>	<i>TC</i>	<i>TA</i>	<i>TC</i>
no Iteration	0	.955	.953	.795	.769	.832	.574
split + merge	1	.962	.956	<u>.803</u>	<u>.782</u>	<u>.807</u>	<u>.567</u>
	3	.966	<u>.960</u>	<u>.823</u>	<u>.792</u>	<u>.842</u>	<u>.587</u>
	5	.962	.961	<u>.815</u>	<u>.789</u>	<u>.841</u>	<u>.588</u>
w/o split	1	.957	.955	.754	.770	.687	.525
	3	.959	.949	.755	.751	.639	.499
	5	.962	.958	.727	.728	.618	.481

```
{% if {n_topic_words} < 1%}
- The topic should be a short phrase of
{{n_topic_words-1}} - {{n_topic_words}} words.
{% endif %}
{% if {n_topic_words} == 1%}
- The topic should be a single word.
{% endif %}
- The description should be no more than two sen-
tences.
- The description needs to start with phrase ‘This
topic is about...’

{% if {topic_definition} is not none %}
{{topic_definition}}
{% endif %}

{% if ({previous_topic} is not none) and
({feedback} is not none) %}
This cluster was previously named as
{{previous_topic}}. Here is some feedback
on how you can improve the quality of the topic:
{{feedback}} {% endif %}

Please keep in mind that the text segments with
different ID does not originated from the same doc-
ument.
```

Text Segments: {{text_segments}}

The response is generated using structured LLM generation, following the pydantic schema shown below.

```
from pydantic import
    BaseModel
    topic_eval_coh

class TopicGenerationOutput (BaseModel)
    topic: str
    description: str
```

B.2. Topic Coherence Evaluation Prompt

We evaluate the coherence of each generated topic, which measures how closely the topic name and description match with the assigned text segments. The prompt presented below is used for this.

Prompt for Topic Coherence Evaluation Agent

You’ll receive a list of text segments, a topic assigned to the given corpus, and its corresponding description.

Your task is to calculate the Topic Coherence score, which evaluates how closely the topic name and the description match with the text segments.

Assign an overall Topic Coherence score from 1 to 10 (including .5 increments), where:

- 10 = High topic coherence: The topic name and description align very well with the text segments.
- 1 = Low topic coherence: The topic name and description covers too broadly compared to the text segments.

If the topic coherence score is sufficient and does not require additional separation:

- specify the field ‘need_separation’ as False.
- leave the fields ‘feedback’ as null.

If the topic coherence score is low and requires separation, please:

- Specify the name of the problematic topic in the ‘topic’ field.
- Specify the field ‘need_separation’ as True.
- Provide detailed feedback in the ‘feedback’ field by addressing the following points:
 - Explain the aspects of the topic and description that do not align well with the text segments.
 - Offer concrete suggestions to improve the coherence between the topic and the text segments.

List of Text Segments: {{text_segments}}

Assigned Topic: {{topic_name}}
Topic Description: {{topic_description}}

The response is generated using structured LLM generation, following the pydantic schema shown below.

```
from pydantic import BaseModel

class TopicCoherenceEvalIndividualOutput(BaseModel):
    topic: str
    score: float
    need_separation: bool
    feedback: str | None
```

B.3. Topic Diversity Evaluation Prompt

Topic diversity evaluation agent performs a holistic topic evaluation, which measures the degree of distinctness among topics—that is, the extent to which they are dissimilar from one another. The prompt presented below is used for this task.

Prompt for Topic Diversity Evaluation Agent

You will receive a list of topics, which are identified by a unique ID, and their corresponding descriptions.

Your task is to calculate the Topic Diversity score, which assesses how distinct the topics are from one another and helps identify if any topics need to be merged or are too similar.

Assign an overall Topic Diversity score from 1 to 10 (including .5 increments), where:

- 10 = High topic diversity: The topics are very distinct from one another.
- 1 = Low topic diversity: Some topics are very similar and may need to be merged.

If the topic diversity score is sufficient and does not require merging:

- specify the field 'need_merge' as False.
- leave the fields 'feedback' and 'topics_to_merge' as null.

If the topic diversity score is low and requires merging, please:

- Specify the field 'need_merge' as True.
- Provide detailed feedback in the 'feedback' field by addressing the following points:
 - Explain which topics are too similar and why they should be merged.
 - Offer concrete suggestions on how to improve topic diversity by merging similar topics.

- For each group of similar topics, list the IDs of the TWO most similar topics to be merged in the 'topics_to_merge' field.

Topics and Descriptions: {{topic_results}}

The response is generated using structured LLM generation, following the pydantic schema shown below.

```
from pydantic import BaseModel

class TopicDiversityEvalOutput(BaseModel):
    score: float
    need_merge: bool
    feedback: str | None
    topics_to_merge: list[list[int]] | None
```

B.4. Planning Agent Prompt

The planner agent first generates summary reports for both before and after the topic refinement iteration by taking the coherence and diversity evaluation results as input and using the following prompt:

Prompt for Summary Report Generation

You will receive a list of topics along with feedback, highlighting areas that need improvement. Your task is to create an evaluation report that summarizes this feedback.

[Instruction]

- Analyze the feedback: Carefully review the comments from evaluators on both topic diversity and topic coherence. Identify key strengths and weaknesses highlighted in the feedback.
- Generate an evaluation report: Based on your analysis, create a concise report that highlights the strengths and weaknesses of the topic modeling results.
- Keep in mind not to include suggestions that are not described in the given feedback.

Please include an overall score in the 'score' field, ranging from 1 to 10, based on the strengths, weaknesses, and individual scores mentioned in the feedback.

Coherence	Evaluation	Results:
{{topic_coherence_eval_results}}		
Diversity	Evaluation	Results:
{{topic_diversity_eval_results}}		

The response is generated using structured LLM generation, following the pydantic schema shown below.

```
from pydantic import BaseModel
```

```
class PlannerEvalReportOutput(BaseModel):
    coherence_score: float
    diversity_score: float
    report_summary: str
```

Once the summary reports are generated, the planning agent determines whether improvements have been achieved by comparing the two. The task is conducted by employing the following prompt:

Prompt for Improvement Check

You will receive two topic modeling evaluation reports: one from before and one from after the current refinement iteration.

Your task is to determine whether there has been an improvement in the topic modeling results based on these reports.

[Instruction]

For making your decision, consider the following aspects:

- 1) Overall Score Comparison: Compare the overall scores from both reports. An improvement is indicated if the post-refinement report has a higher overall score than the pre-refinement report.
- 2) Strengths and Weaknesses: Analyze the strengths and weaknesses highlighted in both reports. Look for areas where the post-refinement report shows fewer weaknesses or improved strengths compared to the pre-refinement report.
- 3) Number of Topics that Need Refinement: Count the number of topics that were identified as needing refinement in both reports. An improvement is indicated if the post-refinement report has fewer topics requiring refinement than the pre-refinement report.
- 4) Specific Feedback Analysis: Review the specific feedback provided for each topic in both reports. Check how severe the issues were in the pre-refinement report and whether they have been adequately addressed in the post-refinement report. Look for positive changes or resolved issues in the post-refinement report compared to the pre-refinement report.

If the improvement criteria are met, indicate 'is_improved' as 'True'. Otherwise, indicate 'is_improved' as 'False'. Also, provide a brief explanation of your decision in 'reason' field.

Pre-refinement Report: {{report_before}}

Post-refinement Report: {{report_after}}

The response is generated using structured LLM generation, following the `pydantic` schema shown below.

```
from pydantic import BaseModel

class PlannerImprovementCheckOutput(
    BaseModel):
    is_improved: bool
    reason: str
```

B.5. LLM-as-a-Judge Prompt

To evaluate topic accuracy ($\mathcal{T}A$) and topic completeness ($\mathcal{T}C$) using LLM-as-a-Judge, we used the following prompts:

Prompt for Topic Accuracy Evaluation

We will show you a model-generated topics, which are assigned to the given text.

Your task is to evaluate the accuracy of the generated topic over the given text; how accurate the topic is and well represented in the given text.

You can ignore the level of granularity / detail. Please just assess if the topic is accurate given the text in any level of detail.

You should evaluate the topic accuracy in four level: Incorrect, Partially Correct, Mostly Correct, and Completely Correct. The definitions of each level are as follows:

- Incorrect: The details in the topic is not represented in the text in any way. The topic label is completely wrong or directly contradicting the text.
- Partially Correct: The details of the topic is partially represented in the text. There is some relevance of the topic label to the contents in the text.
- Mostly Correct : The details of the topic is mostly represented in the text.
- Completely Correct : All the details of the topic are represented in the text.

Text: {{text}}

Assigned Topic: {{topic}}

Prompt for Topic Completeness Evaluation

We will show you a model-generated topic(s), which are assigned to the given text.

Your task is to evaluate the completeness of the topic, which assesses whether there exist any other topics that are present in the given text but missing.

You should evaluate the topic completeness in four level: Not Covered, Minorly Covered, Mostly Covered, and Complete. The definitions of each level

are as follows:

- Not Covered: The assigned topic(s) do not reflect any of the main themes or subjects present in the text. Significant topics are missing, and the model's output fails to capture the core content.
- Minorly Covered: The assigned topic(s) capture only a small portion of the relevant themes in the text. Most important topics are missing, and the coverage is insufficient to represent the text's content.
- Mostly Covered: The assigned topic(s) capture most of the main themes or subjects in the text, but one or more significant topics are still missing. The coverage is substantial but not fully complete.
- Complete: The assigned topic(s) fully reflect all major themes and subjects present in the text. No significant topics are missing, and the model's output provides a comprehensive representation of the text.

Text: {{text}}

Assigned Topic: {{topic}}

The responses of both evaluation dimensions are generated using structured LLM generation, following the `pydantic` schema shown below.

```
from pydantic import BaseModel

class AutoEvalOutput(BaseModel):
    answer: str
    reason: str
```

C. Examples

C.1. Public Dataset Examples

Table 8. Examples of public datasets and their corresponding topics generated by SELATM.

Dataset	Input Text	Generated Topic
Banking77	The exchange rate applied is wrong for my international purchase.	{‘Incorrect Exchange Rates’: 1.0}
Bills	Restore American Dream Act of 2007 - Amends the Internal Revenue Code to establish tax-exempt homeownership plans. Allows a tax deduction from gross income for cash contributions to such plans. Defines "homeownership plan" as a trust established for the exclusive purpose of paying the costs (e.g., downpayment, interest, mortgage insurance, closing costs, etc.) of acquiring a principal residence by an individual who has never owned a principal residence. Excludes from gross income distributions from a homeownership plan used to pay the costs of acquiring a principal residence. Sets forth rules governing: (1) transfers of an interest in a plan due to death or divorce; (2) penalties for making distributions from a plan for purposes other than to acquire a principal residence; and (3) employer homeownership plans established for the benefit of their employees.	{‘Housing Legislation’: 0.2, ‘Tax Legislation’: 0.8}
Wiki	Borodino-class battlecruiser = The Borodino-class battlecruisers were a group of four battlecruisers ordered by the Imperial Russian Navy before World War I. Also referred to as the Izmail class , they were laid down in December 1912 at Saint Petersburg for service with the Baltic Fleet. [...] None of the proposals was accepted, and all three of the less complete ships were sold to a German company for scrap on 21 August 1923 to raise much-needed cash for the government . The Operational Administration of the Soviet Navy worked out the requirements in May 1925 for a conversion that would have made Izmail into an aircraft carrier with a top speed of 27 knots (50 km / h;31 mph) and a capacity of fifty aircraft. She would have been armed with eight 183-millimetre (7.2 in) guns and her armour reduced to a maximum of 76 millimetres (3.0in). This proposal was approved by Alexey Rykov , Chairman of the Council of the People ’s Commissars on 6 July 1925. The plan was cancelled on 16 March 1926 after the Red Army managed to gain control of a commission appointed to review the needs of the Navy, and the Army was strongly opposed to spending additional money on naval projects. She was scrapped beginning in 1931 in Leningrad . = = Ships = = The ships were named after battles fought by the Russian Empire:	{‘Naval Warfare’: 0.517, ‘Naval Armament’: 0.379, ‘Naval Propulsion’: 0.103}

Table 9. The descriptions of topics presented in Table 8.

Topic	Generated Topic Description
Incorrect Exchange Rates	This topic is about issues related to incorrect exchange rates applied during purchases or cash withdrawals in foreign currencies. Customers are expressing concerns about discrepancies between expected and actual exchange rates applied to their international transactions.
Housing Legislation	This topic is about various legislative acts and amendments related to housing policies, including public housing, tax exemptions for homeownership, mortgage regulations, and housing assistance programs. It encompasses issues such as rent adjustments, affordable housing eligibility, support for vulnerable populations, and regulatory simplifications to improve housing access and management.
Tax Legislation	This topic is about various legislative amendments to the Internal Revenue Code aimed at modifying tax rates, exclusions, and rules related to corporate, estate, and investment taxation. These amendments address issues such as estate and gift tax exclusions, corporate expatriations, timber gains, and regulated investment companies, reflecting ongoing efforts to refine tax policies and their economic implications.
Naval Warfare	This topic is about naval engagements and operations during various wars, focusing on the activities, missions, and modifications of naval ships and fleets. It includes minelaying, escort duties, battles, and ship modifications across different time periods and conflicts.
Naval Armament	This topic is about the various armaments and defensive features used in naval ships across different eras, including guns, torpedoes, and armor configurations.
Naval Propulsion	This topic is about the propulsion systems and specifications of various naval ships, detailing their power sources, engine types, speeds, fuel capacities, and ranges. It encompasses the technological aspects of how these ships were designed to achieve their operational capabilities and endurance at sea.

C.2. Business Dataset Examples

Table 10. Examples of business datasets and their corresponding topics generated by SELATM.

Dataset	Input Text	Generated Topic
CCC	VIRTUAL_AGENT : Hi [Customer_Name]. How can I help you? CUSTOMER : Hi, how to avoid overdrawn fee VIRTUAL_AGENT : Which account would you like to get info on fees for? We want to make sure you're not surprised by a fee you didn't expect. VIRTUAL_AGENT : Here's where you can learn about the fees associated with your account, plus tips on avoiding some of them.	{ 'Chatbot Greetings': 0.25, 'Overdraft Fees': 0.25, 'Fee Notifications': 0.25, 'Fee Information': 0.25 }
BCR	I kept getting the same response to an issue different than the one I'm asking about	{ 'Repetitive Responses': 1.0 }
CI	Customer noticed a \$109 charge on their Apple bill pay for an in-person transaction in California, which they did not recognize.	{ 'Apple Pay Issues': 1.0 }

Table 11. The descriptions of topics presented in Table 10.

Topic	Generated Topic Description
Chatbot Greetings	This topic is about the variety of introductory messages used by the bank's virtual agent to greet customers and initiate conversations, highlighting the personal touch in customer service interactions.
Overdraft Fees	This topic is about customers' concerns and inquiries regarding overdraft fees, including requests for fee reversals, waivers, protection, and clarifications on charges. Customers often seek assistance to avoid or dispute these fees and express dissatisfaction with unexpected charges.
Fee Notifications	This topic is about the bank's virtual agent ensuring that customers are informed about fees that might appear on their accounts, preventing any unexpected charges. It reflects the bank's proactive approach in maintaining transparency with its clients regarding potential fees.
Fee Information	This topic is about providing information on bank account fees and offering advice on how to avoid them. It focuses on educating customers about the costs associated with their accounts and helping them manage or reduce potential charges.
Repetitive Responses	This topic is about users experiencing frustration with the chatbot providing repetitive and unhelpful responses, requiring them to rephrase questions multiple times without improvement in the outcome.
Apple Pay Issues	This topic is about various difficulties customers face with Apple Pay, including unauthorized charges, declined transactions, and problems with account activation or verification.